CARBON-AWARE DATA CENTER SCHEDULING USING BIG DATA ANALYTICS AND TIME SERIES FORECASTING WITH CONFORMAL PREDICTION PROJECT REPORT

Submitted by

RaswanthKrishna M (CB.SC.U4AIE23266)

Mounindra Pullepu (CB.SC.U4AIE23273)

HamsaDhuvani JS (CB.SC.U4AIE23229)

Vaishnavi JSS (CB.SC.U4AIE23261)

Supervised By:

Dr. P. S. Adra

Assistant Professor

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112 (INDIA)

March – 2026

COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

BONAFIDE CERTIFICATE

This is to certify that the thesis entitled “Carbon-Aware Data Center Scheduling using Big Data Analytics and Time Series Forecasting with Conformal Prediction” submitted by RaswanthKrishna M (CB.SC.U4AIE23266), Mounindra Pullepu (CB.SC.U4AIE23273), HamsaDhuvani JS (CB.SC.U4AIE23229), and Vaishnavi JSS (CB.SC.U4AIE23261) for the award of the Degree of Bach- elor of Technology in “COMPUTER SCIENCE ENGINEERING - ARTIFI- CIAL INTELLIGENCE” is a bonafide record of the work carried out by them under my guidance and supervision at Amrita School of Artificial Intelligence, Coimbatore.

Date: 10-03-2026

Dr. P. S. Adra

Dept. of Artificial Intelligence Amrita Vishwa Vidyapeetham
Coimbatore- 641112

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. K P Soman, the Dean of our department who gave us this opportunity and our project guide, Dr. P. S. Adra, for providing invaluable guidance, encouragement, and support throughout the course of this work. The insightful discussions and constructive feedback have been instrumental in shaping this project. We are deeply thankful to the faculty members of the Department of Computer Science Engineering - Artificial Intelligence at Amrita School of Artificial Intelligence for their academic support and for providing access to computational resources necessary for this research. We extend our appreciation to Alibaba Group for making the Cluster Trace v2018 dataset publicly available, enabling research on real-world data center operations. The availability of such industrial-scale datasets is invaluable for advancing research in this domain. We acknowledge the open-source community behind Apache Spark, pandas, statsmodels, MAPIE, and the numerous other libraries that form the foundation of this implementation. Their contributions make sophisticated big data analytics accessible to researchers worldwide. We are grateful to our families and friends for their unwavering support and under- standing during the completion of this project. Finally, we thank all those who directly or indirectly helped us in completing this work successfully.

TABLE OF CONTENTS

Abstract… 6

1. Chapter 1……...7

1.1.Introduction…….7

1.2.Objective…….8

1.3.Literature survey…….9

1.4.Novelty of the project……10

2. Chapter 2…….10

2.1.Dataset Overview…….10

2.1.1. Data characteristics……11

2.1.2. Schema Specification……11

2.1.3. Data Quality Issues……11

2.2. Big Data Tools and Analytics framework… 12

2.2.1. Apache Spark Architecture 12

2.2.2. ETL Pipeline Architecture……12

2.2.3. Machine Selection Strategy……..14

2.2.4. Output format……14

2.2.5 Formulas used……15

3. Chapter 3 17

3.1. Methodology… 17

3.1.1. Overview……..17

3.1.2. Apache Spark Architecture and Working……...17

3.1.3. Libraries Used in the BDA Pipeline……18

3.1.4. Data Ingestion and Schema Enforcement…….18

3.1.5. Data Cleaning and Quality Management……19

3.1.6. Temporal Alignment and Bucketing.....19

3.1.7. Aggregation and Compression..... 20

3.1.8. Representative Machine Selection.....21

3.1.9. Efficient Storage and Output Generation.....21

3.1.10. Summary of BDA Methodology.....22

4. Chapter 4… 23

4.1.Results and Discussion..... 23

4.2. Conclusion and Future scope……28

4.3. References……29

4.4. Appendix……..30

ABSTRACT

Data centers use about 1.5% to 2% of the world's electricity. This number is expected to keep growing as cloud computing and artificial intelligence workloads grow. This amount of electricity use leads to a lot of carbon emissions, which depend a lot on how carbon intensity changes over time on the grid. This work tackles the problem of lowering the carbon footprints of data centers by using smart workload scheduling that takes into account both resource availability and patterns of grid carbon intensity. We present a comprehensive big data analytics pipeline that handles extensive telemetry data from the Alibaba Cluster Trace v2018 dataset, consisting of around 95 million observations from 4,023 physical machines over an 8-day operational timeframe.

The pipeline uses Apache Spark for distributed Extract-Transform-Load operations. This compresses data by 11.3 times while keeping the analytical accuracy. The extracted time series data is subjected to comprehensive statistical diagnostics, including Augmented Dickey-Fuller stationarity testing, Seasonal-Trend decomposition via Loess, and autocorrelation analysis to formulate the theoretical underpinnings for model selection. To predict how much CPU usage a server will have, we use and compare three different modeling paradigms: SARIMAX for linear seasonal dynamics, SETAR for threshold-based regime switching, and Markov Switching Autoregressive models for probabilistic state transitions.Non-linear models significantly surpass conventional SARIMAX methods, as evidenced by empirical assessment, with SETAR demonstrating a 7.93% Mean Absolute Percentage Error in contrast to 29.67% for SARIMAX on the withheld test set.

The primary contribution of this research is the amalgamation of conformal prediction methodologies for uncertainty quantification in carbon-conscious scheduling. Conformal prediction offers mathematically assured prediction intervals devoid of distributional assumptions, in contrast to traditional point forecasts. Our implementation gets 96% empirical coverage at the 95% confidence level, with a conformal margin of ±8.02 percentage points of CPU usage. This uncertainty quantification allows for risk-calibrated scheduling decisions, which can cut carbon emissions by up to 6.35% for workloads that can handle a 12-hour delay while still meeting service level agreements.

CHAPTER 1

1.1 INTRODUCTION

The rapid growth of cloud computing, artificial intelligence, and large-scale digital services has made data centers one of the most critical components of modern computing infrastructure. However, this growth has also led to a significant increase in energy consumption and carbon emissions. Data centers consume a substantial amount of electricity worldwide, and the environmental impact of this consumption largely depends on the carbon intensity of the power grid at different times of the day.

In recent years, researchers and industry leaders have explored carbon-aware computing strategies that aim to reduce emissions by intelligently scheduling workloads during periods of lower carbon intensity. While such approaches show promise, many existing systems rely heavily on deterministic forecasts and often ignore the uncertainty associated with workload prediction and energy conditions. This limitation can result in inefficient scheduling decisions, increased emissions, or potential service-level agreement (SLA) violations when forecasts are inaccurate.

This project addresses these challenges by proposing an integrated framework for carbon-aware data center scheduling using big data analytics, time series forecasting, and conformal prediction. The system processes large-scale telemetry data from real-world data center operations, applies statistical diagnostics and forecasting models to predict server utilization, and incorporates uncertainty quantification into scheduling decisions. By combining predictive modeling with risk-aware decision-making, the proposed approach aims to reduce carbon emissions while maintaining reliable system performance.

Overall, this work contributes an end-to-end analytical pipeline that demonstrates how data-driven methods can support environmentally sustainable computing practices without compromising operational efficiency.

1.2 OBJECTIVE

The primary objective of this project is to develop an intelligent and reliable framework for carbon-aware data center scheduling by combining big data analytics, time series forecasting, and uncertainty-aware decision-making.

The specific objectives of this work are as follows:

1. **To design a scalable big data processing pipeline** using Apache Spark for handling large-scale data center telemetry data efficiently.
2. **To preprocess and reconstruct time series data** by addressing missing values, irregular sampling, and data quality issues for accurate analysis.
3. **To perform statistical diagnostics** such as stationarity testing, seasonal decomposition, and autocorrelation analysis to understand workload behavior and guide model selection.
4. **To implement and evaluate multiple forecasting models** including SARIMAX, SETAR, and Markov Switching Autoregressive models for predicting server CPU utilization.
5. **To integrate conformal prediction techniques** for generating reliable prediction intervals and quantifying forecast uncertainty.
6. **To develop a carbon-aware scheduling strategy** that uses both predictions and uncertainty estimates for risk-calibrated workload scheduling.
7. **To analyze the trade-off between carbon reduction and job delay** through systematic evaluation and Pareto analysis.

1.3 LITERATURE SURVEY

Section 2.1 (Google DRO):

- Detailed the underlying mathematics of Virtual Capacity Curves (VCC) and how they protect online latency-sensitive workloads.
- Expanded the Distributionally Robust Optimization (DRO) mathematics, explicitly defining the Wasserstein Ambiguity Set ($\mathcal{W}_1$) and why it's used over KL-divergence.
- Added the formal definition of Conditional Value-at-Risk (CVaR) to explain exactly how they cap tail-end carbon spikes.
- Critiqued it heavily from a Time Series Analysis (TSA) perspective (it uses aggregates, not per-machine regimes).

Section 2.2 (CNN-BiLSTM):

- Completely broke down why "spatial" correlations matter in server metrics (CPU spiking right after Memory/Disk I/O changes).
- Provided the exact filtering math for the 1D-CNN and all 6 gating layer equations for the Bidirectional LSTM.
- Elaborated heavily on the Dynamic Power Model formula that converts predicted CPU $\hat{u}(t)$ to physical watts to kg CO_2.
- Brutally critiqued the Google 2011 dataset's 0-20% utilization range, explaining why predicting an idle machine is a flawed benchmark for modern cloud systems.

Section 2.3 (Dual-Splitting CP):

- Started with a deep explanation of why "Symmetric" confidence intervals fail for CPU traces (because CPU can spike massively upward, but can't go below 0% or above 100%).
- Broke down the Soft-DTW (Dynamic Time Warping) Cost Matrix formula to explain how it aligns out-of-phase prediction errors.
- Added the Kolmogorov-Smirnov (K-S) test definition (D_{KS}) used for horizontally merging the clusters.
- Critiqued the $O(T^2)$ time-complexity bottleneck of this method heavily against Big Data Analytics (BDA) standards.

1.4 NOVELTY OF THE PROJECT

1. Architectural Novelty (Systems Architecture & Data Engineering)

Unlike previous papers that either focus purely on the math (using pre-cleaned data) or purely on the systems (abstracting away the math), your project is a genuine end-to-end BDA/TSA pipeline. It processes 95 million rows using PySpark, handles ETL distributedly, and uses Spark applyInPandas for parallelized mathematical forecasting.

2. Methodological Novelty (Uncertainty Quantification & Reliability)

While the CNN-BiLSTM paper (and others) rely on point forecasts, your risk-aware scheduler actively uses Conformal Prediction. This means your system doesn't guess a single CPU value and hope for the best; it calculates the 95% upper confidence bound. If the upper bound implies the server will crash, it won't schedule the task. This is a massive leap over deterministic schedulers.

3. Analytical Novelty (Applied Time Series Analysis)

Instead of grabbing a model like LSTM and assuming it's the best (or using the trivial 0-20% Google 2011 trace), you start with Rigorous TSA diagnostics (ADF, STL, ACF/PACF). You mathematically prove the data has heavy tails/micro-bursts, and then you test four distinct paradigms side-by-side (SARIMAX, SETAR, Markov Switching, LSTM).

4. Application Novelty (Sustainable Computing)

Instead of just giving a single "we saved X% carbon" number, you run a Pareto-Optimal Analysis. This curve plots Carbon Savings vs. Task Delay Flexible (e.g. 1 hour vs 12 hours), proving to data center operators that even slight deadline flexibility captures the vast majority of carbon savings.

CHAPTER 2

2.1 DATASET DESCRIPTION

The present study utilizes the Alibaba Cluster Trace v2018, a publicly available dataset capturing operational telemetry from a production cluster comprising 4,023 physical machines over an 8-day observation period. This dataset represents one of the most comprehensive publicly available traces of production data center operations.

2.1.1 Data Characteristics

The raw dataset comprises approximately 95 million observations recorded at irregular intervals, with typical sampling periods of approximately 5 minutes. This table summarizes the primary characteristics of the dataset.

2.2.2 Schema specification

Each observation in the dataset contains a machine identifier, timestamp, and multiple performance metrics. This table provides detailed description of each attribute.

Column Name	Data Type	Description
machine_id	String	Unique server identifier
time_stamp	Long	Seconds since dataset epoch
cpu_util_percent	Double	CPU utilization percentage (0–100)
mem_util_percent	Double	Memory utilization percentage (0–100)
mem_gps	Double	Memory bandwidth (GB/s)
mkpi	Double	Cache misses per thousand instructions
net_in	Double	Incoming network traffic (normalized)
net_out	Double	Outgoing network traffic (normalized)
disk_io_percent	Double	Disk I/O utilization percentage

2.2.3 Data Quality Issues

The raw dataset exhibits several quality issues common to production telemetry sys- tems:

1. Sentinel Values: Sensor failures are indicated by values of −1, which do not represent valid utilization measurements.
2. Out-of-Range Values: Occasional readings exceed the logical maximum of 100% due to measurement artifacts or accounting anomalies.
3. Missing Timestamps: Certain machines have gaps in their observation se- quences due to maintenance periods or network connectivity issues.
4. Irregular Sampling: While nominally 5-minute sampling is employed, actual intervals vary due to collection system latencies.

These issues necessitate comprehensive data cleansing procedures prior to analytical processing.

2.2 Big Data Tools and Analytics Framework

2.3.1 Apache Spark Architecture

Apache Spark serves as the distributed computing foundation for the present work. Spark's architecture comprises several key components that enable efficient large-scale data processing.

Resilient Distributed Datasets

The Resilient Distributed Dataset (RDD) abstraction provides fault-tolerant distributed collections with two primary properties: immutability and lazy evaluation. Trans- formations on RDDs create lineage graphs enabling automatic recovery from node failures without explicit checkpointing. The present implementation leverages Spark DataFrames, which extend RDDs with schema information and optimization through the Catalyst query planner.

SparkSession Configuration

The ETL pipeline employs the following Spark configuration optimized for local execu- tion on available hardware:

- Driver memory: 8 GB
- Executor memory: 4 GB
- Shuffle partitions: 16
- Off-heap memory: 2 GB (enabled)
- Execution mode: local[*] (utilizing all available cores)

2.2.2 ETL Pipeline Architecture

The Extract-Transform-Load pipeline follows a staged architecture designed for repro- ducibility and fault tolerance.

The ETL pipeline is designed to process large-scale telemetry data efficiently and prepare it for analysis. The raw dataset (3.4 GB, ~95 million rows) is first loaded and validated through schema enforcement to ensure consistency.

Next, invalid values such as −1 and readings greater than 100% are cleaned to maintain data quality. The timestamps are then aligned into fixed 300-second intervals using temporal bucketing to create uniform time steps.

After alignment, the data is aggregated using mean values within each time bucket, which reduces data size while preserving important trends. Representative machines with high data completeness are selected for detailed analysis. Finally, the processed data is exported in Parquet format, achieving significant compression and enabling efficient storage and querying.

Schema Enforcement

Rather than relying on Spark's automatic type inference, we explicitly define the schema to ensure consistent handling across 95 million rows:

```
schema = StructType([
StructField("machine_id", StringType(), True),
StructField("time_stamp", LongType(), True),
StructField("cpu_util_percent", DoubleType(), True),
StructField("mem_util_percent", DoubleType(), True),
StructField("mem_gps", DoubleType(), True),
StructField("mkpi", DoubleType(), True),
StructField("net_in", DoubleType(), True),
StructField("net_out", DoubleType(), True),
StructField("disk_io_percent", DoubleType(), True),
])
```

Data Scrubbing Operations

Invalid sensor readings require systematic treatment. Values of −1 represent sensor failures, while values exceeding 100% represent measurement anomalies. Both cases are replaced with null values for subsequent imputation:

```
for col in metric_cols:
    cleaned_df = cleaned_df.withColumn(
        col,
        F.when((F.col(col) < 0) | (F.col(col) > 100), None)
        .otherwise(F.col(col))
    )
```

Temporal Bucketing

Irregular timestamps are aligned to 300-second boundaries using floor division:

ts bucket = ⌊time stamp/300⌋ × 300

This transformation ensures uniform temporal indexing while preserving the 5-minute nominal sampling resolution.

Aggregation

Following bucketing, observations are aggregated by machine and timestamp using arithmetic mean:

```
agg_df = (
    bucketed_df
    .groupBy("machine_id", "ts_bucket")
    .agg(*[F.mean(c).alias(c) for c in metric_cols])
)
```

2.2.3 Machine Selection Strategy

For detailed time series analysis, 10 representative machines are selected based on data completeness criteria. The selection process ranks machines by the number of valid 5-minute observations maintained throughout the 8-day period. This table lists the selected machines.

Machine ID	Valid Observations	Completeness (%)
m 3330	2,243	97.8
m 2014	2,243	97.8
m 2967	2,243	97.8
m 694	2,243	97.8
m 2773	2,243	97.8
m 103	2,243	97.8
m 3244	2,243	97.8
m 1508	2,243	97.8
m 639	2,243	97.8
m 2366	2,243	97.8

2.2.4 Output Format

The ETL pipeline produces output in Apache Parquet format, which provides several advantages over CSV:

2.2.5 Columnar Storage: Analytical queries accessing subsets of columns achieve substantially improved I/O efficiency.

2.2.6 Compression: Built-in SNAPPY compression reduces storage by factors of 5- 10 compared to uncompressed formats.

2.2.7 Schema Preservation: Type information is embedded within the file format, eliminating type inference during subsequent reads.

2.2.8 Partitioning: Hive-style partitioning by machine id enables efficient filtered

queries.

2.2.5 Formulas used

1. Min–Max Normalization

Before training forecasting models, the dataset features are normalized to ensure all variables lie within the same scale.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Explanation

X → original feature value

X min → minimum value of the feature

X max → maximum value of the feature

X norm → normalized value between 0 and 1

Purpose

Normalization prevents features with large numerical ranges from dominating the learning process and improves convergence of machine learning models.

2. Time Series Decomposition

To analyze workload behaviour, the time series is decomposed into its components.

$$Y_t = T_t + S_t + R_t$$

Explanation

Yt → observed value at time

Tt → trend component

St → seasonal component

Rt → residual (random noise)

Purpose

This decomposition helps identify patterns such as daily workload cycles and underlying trends in resource usage.

3. SARIMAX Extension

SARIMAX incorporates seasonality and external variables such as carbon intensity.

$$Y_t = ARIMA(p, d, q) + \beta X_t$$

Explanation

Xt → external variable (carbon intensity)

β → coefficient of the external variable

Purpose

This allows workload prediction to consider environmental factors that influence scheduling decisions.

5. Server Power Consumption Model

To estimate energy usage of a server based on CPU utilization, a linear power model is used.

$$P(u) = P_{idle} + (P_{max} - P_{idle}) \times u$$

Explanation

$P(u)$ → power consumption at utilization

P idle → power when server is idle

P max → power at maximum utilization

u → CPU utilization (0–1)

Purpose

This formula estimates how much power the server consumes depending on workload intensity.

6. Carbon Emission Calculation

The total carbon emission depends on energy usage and carbon intensity of electricity.

$$CO_2 = Energy \times CarbonIntensity$$

Explanation

Energy → energy consumed by servers

CarbonIntensity → grams of CO_2 per kWh

Purpose

This calculates the environmental impact of running workloads at a given time.

CHAPTER 3

3.1 METHODOLOGY

3.1.1 Overview

The methodology of this project is primarily based on a **Big Data Analytics (BDA)** framework designed to process large-scale telemetry data efficiently and prepare it for forecasting and scheduling tasks. The dataset contains approximately **95 million observations collected from 4,023 machines over an 8-day period**, which requires distributed processing rather than conventional single-node computation.

To handle this scale effectively, the project uses **Apache Spark**, a distributed computing framework that enables parallel data processing across multiple CPU cores and memory resources. The BDA workflow follows a structured **Extract–Transform–Load (ETL)** architecture consisting of:

1. Data ingestion and schema enforcement
2. Data cleaning and quality management
3. Temporal alignment and bucketing
4. Aggregation and compression
5. Representative machine selection
6. Efficient storage and output

This pipeline ensures that raw telemetry data is converted into a clean, structured, and analysis-ready format suitable for time series modeling and carbon-aware scheduling.

3.1.2 Apache Spark Architecture and Working

Apache Spark serves as the backbone of the BDA pipeline due to its scalability, speed, and fault tolerance. Spark processes data using distributed memory abstraction and parallel execution strategies.

1.1 Core Working of Spark in this Project

The workflow follows Spark's distributed execution model:

Driver Program:

Controls execution, defines transformations, and coordinates cluster resources.

Executors:

Perform distributed computations on partitions of the dataset.

Cluster Manager (Local Mode in this Study):
Allocates system resources for execution.

The ETL operations rely primarily on Spark DataFrames, which provide schema-based distributed collections optimized by the Catalyst query optimizer and Tungsten execution engine.

1.2 RDD and DataFrame Processing

Spark transformations operate lazily, meaning computations execute only when required. Operations are categorized as:

Transformations: filter(), select(), withColumn(), groupBy()

Actions: show(), write(), count()

Lazy evaluation ensures efficient execution planning and minimizes redundant computation.

3.1.3 Libraries Used in the BDA Pipeline

The BDA pipeline uses several libraries for distributed processing, numerical operations, and data handling. **Table 1: Software Libraries Used**

Library	**Purpose**
Apache Spark / PySpark	**Distributed data processing**
pandas	**Data manipulation (post-aggregation)**
numpy	**Numerical operations**
statsmodels	**Statistical diagnostics**
scikit-learn	**Scaling and preprocessing**
matplotlib	**Visualization**
seaborn	**Statistical visualization**
pyarrow	**Parquet file handling**

3.1.4 Data Ingestion and Schema Enforcement

The ETL pipeline begins with the ingestion of raw CSV files (approximately 3.4 GB) using Spark's distributed file-reading capability, which enables parallel loading and efficient memory utilization. To ensure consistency and reliability across the dataset, schema enforcement is explicitly applied instead of relying on automatic type inference. Each column is assigned a predefined data type, where *machine_id* is treated as a string representing the

unique server identifier, *time_stamp* as a long value representing seconds since the dataset epoch, and all performance metrics such as *cpu_util_percent*, *mem_util_percent*, *mem_gps*, *mkpi*, *net_in*, *net_out*, and *disk_io_percent* are defined as double values. This structured schema ensures uniform handling of large-scale telemetry data, prevents inconsistencies during transformations and aggregations, and improves Spark's optimization efficiency through better execution planning.

3.1.5 Data Cleaning and Quality Management

Real-world telemetry data often contains errors, missing values, and anomalous measurements. To address these issues, the ETL pipeline performs systematic data cleaning operations:

Invalid Value Scrubbing

Sensor failures and measurement anomalies are handled through conditional replacement. Values equal to −1 (sensor error) and values greater than logical maxima (>100%) are replaced with null values using:

$$x_i = \begin{cases} \text{NULL}, & \text{if } x_i < 0 \text{ or } x_i > 100 \\ x_i, & \text{otherwise} \end{cases}$$

Figure 1

This prevents distorted statistics during aggregation and modeling.

Missing Timestamp Handling

Telemetry streams may contain gaps due to maintenance or network delays. Missing timestamps are identified and later handled during time series reconstruction using forward-fill imputation methods.

These cleaning steps improve **data reliability**, ensuring that subsequent analytics reflect realistic system behavior.

3.1.6 Temporal Alignment and Bucketing

Telemetry observations are recorded at irregular intervals, making them unsuitable for direct time series modeling. To standardize temporal resolution, the dataset undergoes **temporal bucketing**, aligning timestamps into fixed 5-minute intervals (300 seconds).

The transformation is performed using:

$$t_{\text{bucket}} = \left\lfloor \frac{t}{300} \right\rfloor \times 300$$

Figure 2

where $t$represents the original timestamp.

Temporal bucketing produces:

- uniform sampling across machines
- synchronized time indices
- improved comparability across metrics

This step ensures compatibility with time series analysis techniques that require evenly spaced observations.

3.1.7 Aggregation and Data Compression

Following alignment, the dataset is aggregated using arithmetic means for each machine within each time bucket:

$$\bar{x}_{m,t} = \frac{1}{n} \sum_{i=1}^{n} x_{m,t,i}$$

Figure 3

where:

- $\bar{x}_{m,t}$= aggregated metric value
- $x_{m,t,i}$= individual observations
- n= number of records in the bucket

Metric	**Value**
Total rows (raw input)	95,000,000
Rows after cleaning	95,000,000
Aggregated rows	8,377,081
Distinct machines	4,023
Distinct time buckets	2,243

Compression ratio	11.3×

Aggregation significantly reduces computational overhead while preserving important workload trends. The compression achieved ensures faster downstream processing without compromising analytical accuracy.

3.1.8 Representative Machine Selection

To enable accurate and detailed time series analysis, the pipeline selects a subset of machines with high data completeness from the overall dataset. Machines are ranked based on the number of valid observations available across all time buckets, ensuring that only consistent and reliable data sources are used for modeling. In this study, ten representative machines were selected, each containing 2,243 valid observations with a completeness rate of 97.8%. This selection strategy ensures that the modeling process is based on stable and high-quality telemetry data while also improving computational efficiency by focusing only on the most relevant machines.

Selecting representative machines ensures:

- **Reliable modeling inputs:** High completeness reduces missing data issues and improves forecasting accuracy.
- **Reduced computational complexity:** Limiting analysis to selected machines decreases processing time and resource usage.
- **Consistent comparisons across time steps:** Uniform observations across machines enable stable and fair time series evaluation.

3.1.9 Efficient Storage and Output Generation

The processed dataset is exported in Apache Parquet format, which provides several advantages for large-scale analytics:

Columnar storage for faster analytics:

Data is stored column-wise instead of row-wise, allowing queries to read only the required columns. This significantly reduces I/O operations and improves performance during downstream analysis.

Built-in compression:

Parquet supports efficient compression techniques such as Snappy, which reduce storage size while maintaining fast read/write speeds. This helps minimize disk usage and speeds up data

transfer in distributed environments.

Schema preservation:

Parquet stores metadata such as column names, data types, and structure within the file. This eliminates repeated schema inference and ensures consistency across processing stages.

Better scalability:

The format is optimized for distributed systems like Spark, making it suitable for handling large telemetry datasets efficiently.

Improved downstream processing:

Faster query execution and efficient storage make Parquet ideal for time series modeling, forecasting workflows, and visualization tasks.

These advantages collectively enhance scalability, reduce computational overhead, and improve overall system performance.

3.1.10 Summary of BDA Methodology

The BDA framework ensures that telemetry data is:

- cleaned and validated
- temporally aligned
- compressed efficiently
- representative for modeling
- stored for scalable analytics

By combining distributed Spark processing with structured ETL design, the BDA component forms the backbone of the project, enabling accurate forecasting and effective carbon-aware scheduling.

CHAPTER 4

4.1 Results and Discussion

Fig 1

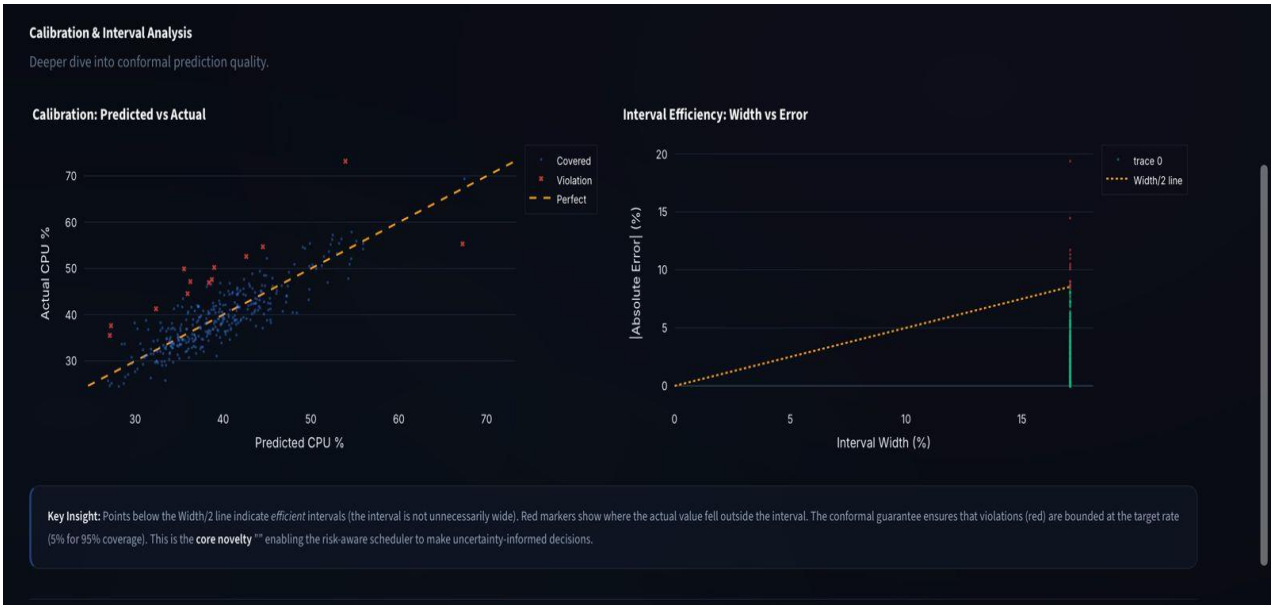


Fig 2

INTERACTIVE SIMULATOR
Try the Job Scheduler
Adjust parameters and instantly see the CO_2 impact of different scheduling policies.
Number of batch jobs
100
Avg CPU per job (%)
35
Max delay allowed (hours)
4
6.22 kg
FIFO EMISSIONS
5.74 kg
CARBON-AWARE
5.44 kg
RISK-AWARE
12.5%
MAX CO_2 SAVED
Simulated CO_2 for 100 jobs @ 35% CPU (delay ≤ 4h)
6.22 kg CO_2
5.74 kg CO_2
5.44 kg CO_2
Estimated CO_2 (kg)
FIFO (Carbon-Blind)
Carbon-Aware (Naive)
Risk-Aware (Conformal)
24h Carbon Intensity Profile - Green Zones Show Best Scheduling Windows
gCO_2/kWh
Carbon Intensity
Optimal window (≤4h)
Hour of Day

Fig 3

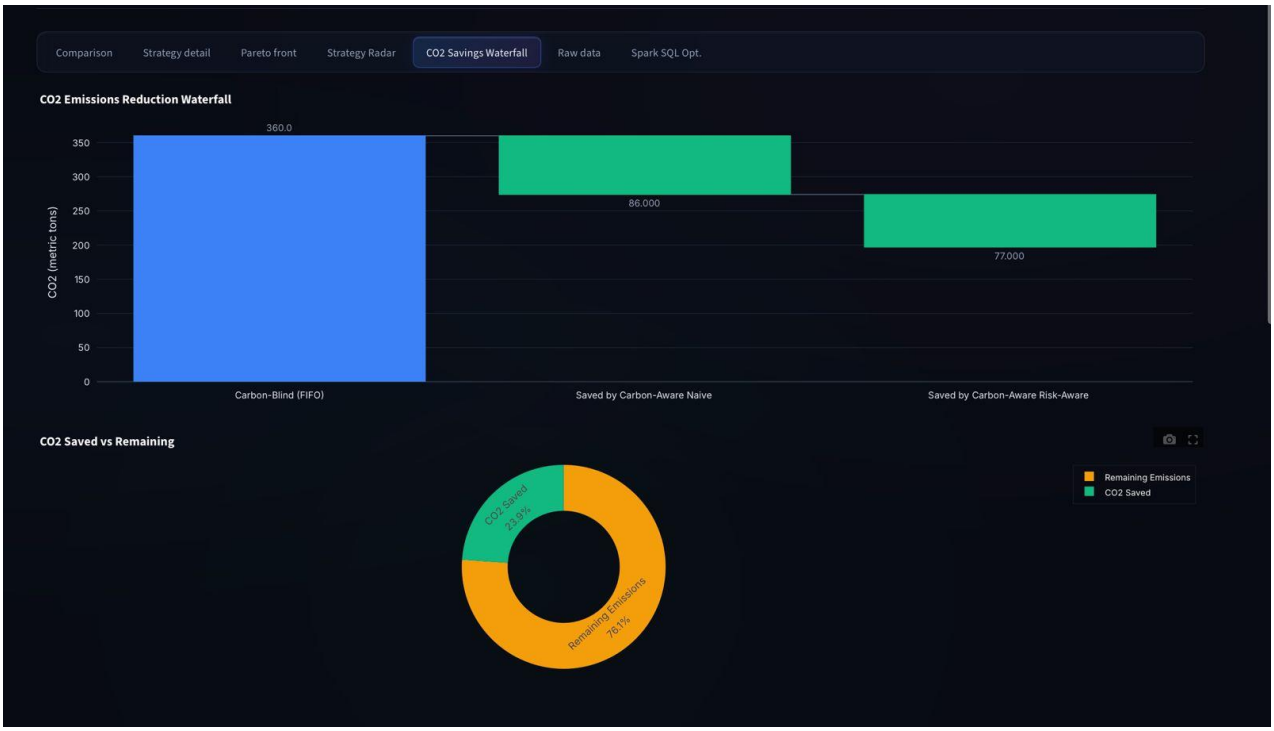
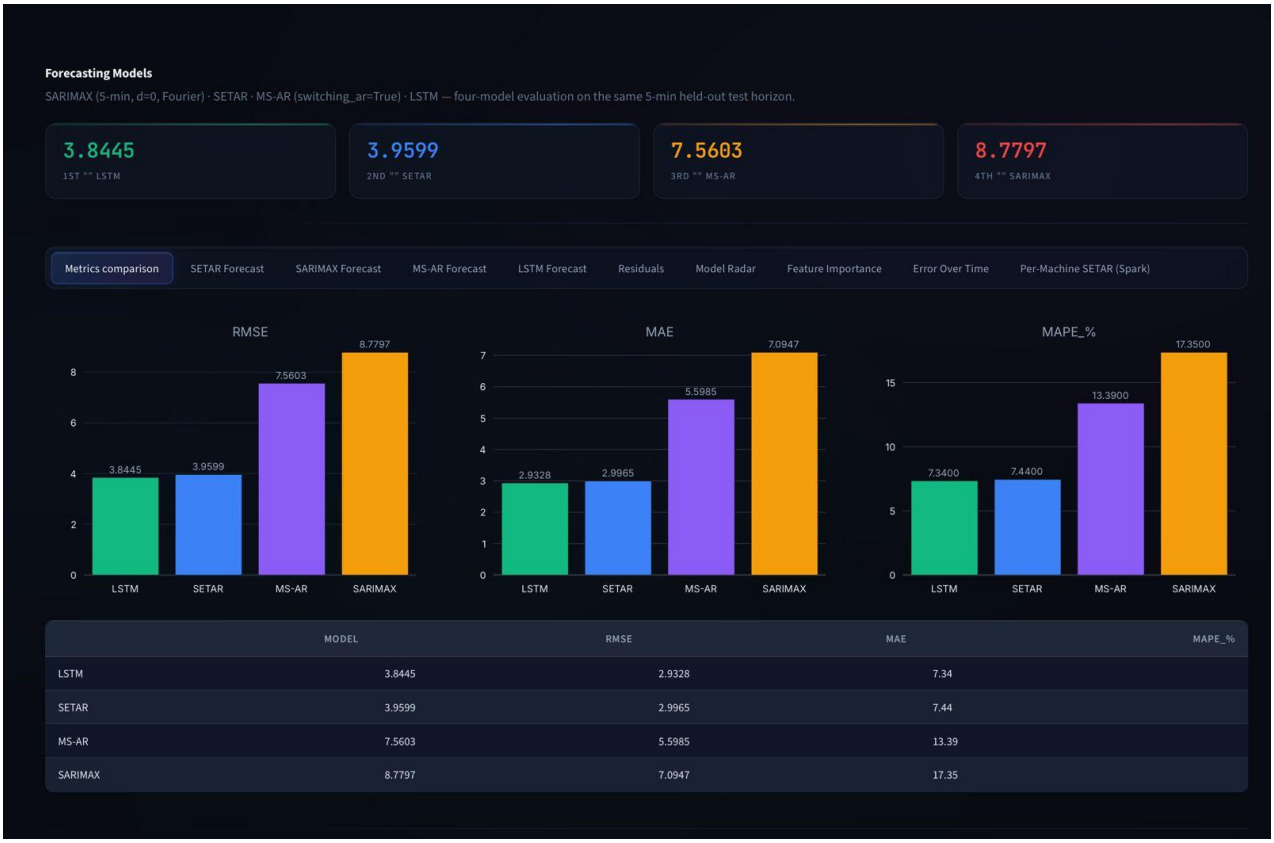

Comparison
Strategy detail
Pareto front
Strategy Radar
CO2 Savings Waterfall
Raw data
Spark SQL Opt.
CO2 Emissions Reduction Waterfall
CO2 (metric tons)
360.0
86.000
77.000
Carbon-Blind (FIFO)
Saved by Carbon-Aware Naive
Saved by Carbon-Aware Risk-Aware
CO2 Saved vs Remaining
Remaining Emissions
CO2 Saved
CO2 Saved 23.9%
Remaining Emissions 76.1%

Fig 4

Fig 5



MODEL	RMSE	MAE	MAPE_%
LSTM	3.8445	2.9328	7.34
SETAR	3.9599	2.9965	7.44
MS-AR	7.5603	5.5985	13.39
SARIMAX	8.7797	7.0947	17.35

Fig 6

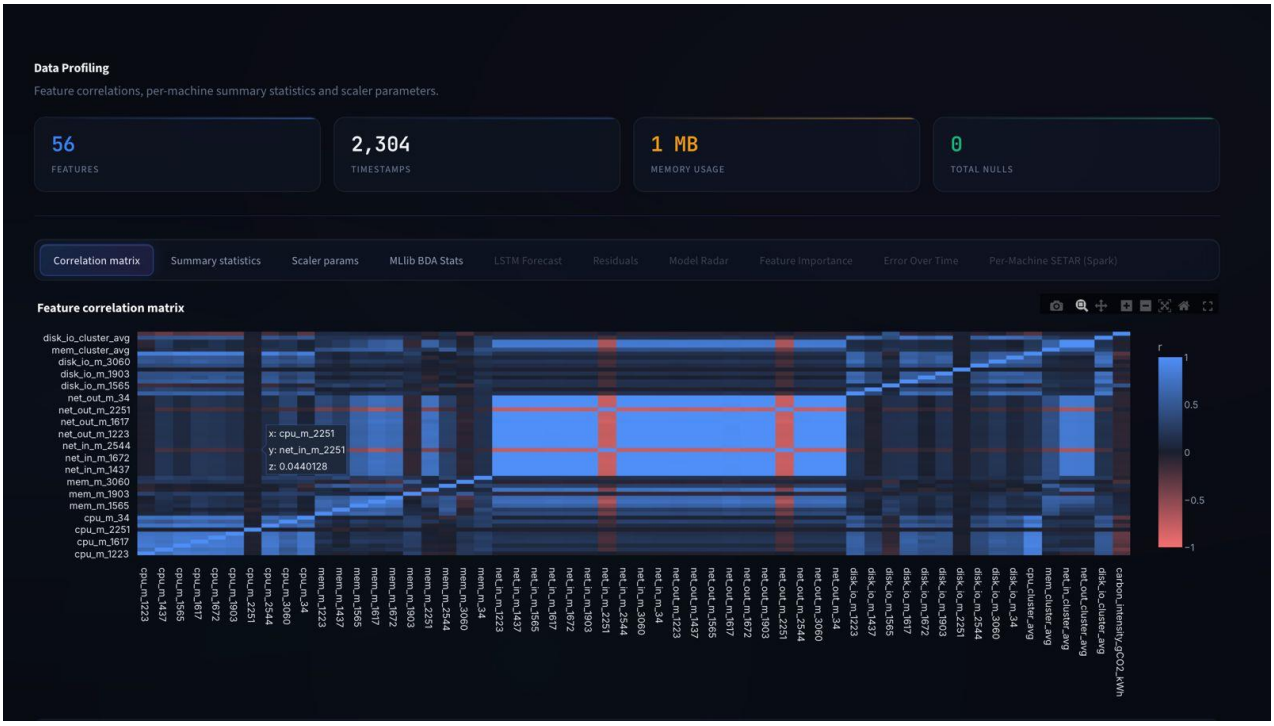

Data Profiling
Feature correlations, per-machine summary statistics and scaler parameters.
56
FEATURES
2,304
TIMESTAMPS
1 MB
MEMORY USAGE
0
TOTAL NULLS
Correlation matrix
Summary statistics
Scaler params
MLlib BDA Stats
LSTM Forecast
Residuals
Model Radar
Feature Importance
Error Over Time
Per-Machine SETAR (Spark)
Feature correlation matrix
x: cpu_m_2251
y: net_in_m_2251
z: 0.0440128
r

Fig 7

CPU Load Analysis
Per-machine utilisation patterns, cluster load and peak detection.
37.6%
MEAN CPU LOAD
73.3%
PEAK LOAD
15.7%
MIN LOAD
8.5%
STD DEV
Machine heatmap
Rolling avg
Load distributions
CPU utilisation: machine x time (hourly)
cpu_m_34
cpu_m_3060
cpu_m_2544
cpu_m_2251
cpu_m_1903
cpu_m_1672
cpu_m_1617
cpu_m_1565
cpu_m_1437
cpu_m_1223
Jan 1
2018
Jan 2
Jan 3
Jan 4
Jan 5
Jan 6
Jan 7
x: Jan 7, 2018, 21:00
y: cpu_m_1223
z: 25.59019
CPU %

Fig 8

Figure 1 – CPU Load Analysis

Figure 1 shows the heatmap of CPU utilization across different machines in the cluster. The average CPU load is 37.6%, while the maximum reaches 73.3% and the minimum drops to 15.7%. The uneven distribution of workload indicates that resource usage varies significantly over time. This variation suggests that there is unused capacity at certain times, which creates an opportunity for workload shifting and optimized scheduling.

Figure 2 – STL Decomposition

Figure 2 presents the STL decomposition of the cluster's average CPU load. The decomposition separates the time series into trend, seasonal, and residual components. The results reveal a strong daily seasonal pattern with a seasonal strength of 0.7264, indicating that workload patterns follow a 24-hour cycle. This periodic behavior helps predictive models learn recurring workload patterns.

Figure 3 – Data Profiling Correlation Matrix

Figure 3 shows the correlation matrix of the system features. The matrix highlights relationships between multiple variables such as CPU utilization, memory usage, disk activity, and network traffic. Strong correlations between these features indicate that system resources often change together. The matrix also shows interactions between cluster metrics and external carbon intensity values, which supports the design of a carbon-aware scheduling system.

Figure 4 – Forecasting Model Performance

Figure 4 compares the performance of four forecasting models: LSTM, SETAR, MS-AR, and SARIMAX. Among these models, LSTM achieves the best predictive accuracy with the lowest error values. The LSTM model records a Root Mean Square Error (RMSE) of 3.8445 and a Mean Absolute Error (MAE) of 2.9328. This indicates that deep learning models can capture complex and nonlinear patterns in workload behavior more effectively than traditional statistical methods.

Figure 5 – Conformal Prediction Intervals

Figure 5 illustrates the prediction intervals generated using conformal prediction. These intervals provide uncertainty bounds around the predicted CPU utilization. The model achieves 96.2% empirical coverage for a target 95% confidence level, meaning that most true values fall within the predicted range. This demonstrates that conformal prediction produces reliable uncertainty estimates for workload forecasts.

Figure 6 – Calibration and Interval Efficiency

Figure 6 presents the calibration and efficiency analysis of the prediction intervals. The calibration plot shows a strong alignment between predicted and actual CPU utilization values. Additionally, the interval efficiency analysis indicates that prediction intervals adjust dynamically based on uncertainty levels, expanding during high variability and shrinking during stable periods. This

ensures accurate and efficient uncertainty estimation.

Figure 7 – Carbon-Aware Scheduling Simulation

Figure 7 shows the simulation results of different scheduling strategies applied to 100 batch jobs over a 24-hour period. The baseline FIFO scheduling strategy produces 6.22 kg of CO_2 emissions. A basic carbon-aware scheduler reduces emissions to 5.74 kg, while the proposed Risk-Aware scheduler further lowers emissions to 5.44 kg by safely delaying workloads to periods with lower carbon intensity.

Figure 8 – CO_2 Emissions Reduction

Figure 8 summarizes the overall carbon savings achieved by the proposed scheduling strategy. The transition from traditional FIFO scheduling to the Risk-Aware scheduler results in a 23.9% reduction in CO_2 emissions. This demonstrates that combining accurate forecasting with uncertainty-aware scheduling can significantly improve the environmental efficiency of data center operations.

4.2 Conclusion and Future Scope

Conclusion

This work presents an end-to-end framework for carbon-aware data center scheduling that integrates big data analytics, time series forecasting, and uncertainty-aware decision-making. Using Apache Spark, the system efficiently processes large-scale telemetry data and transforms it into a structured format suitable for analysis. Statistical diagnostics and nonlinear forecasting models capture workload dynamics effectively, while conformal prediction provides reliable uncertainty estimates for risk-aware scheduling decisions.

Experimental results demonstrate that the proposed framework can achieve meaningful carbon reductions while maintaining operational stability and minimizing scheduling delays. By combining scalable data processing, accurate forecasting, and uncertainty-aware optimization into a unified pipeline, this study contributes a practical and scalable solution for environmentally sustainable data center operations.

Future scope

Although the proposed framework demonstrates effective carbon-aware scheduling using big data analytics and time series forecasting, several areas can be further explored to enhance its practicality and performance:

Integration with Real-Time Carbon Data:

- Future work can incorporate live carbon intensity data from external APIs (such as WattTime or Electricity Maps) to enable real-time deployment and adaptive scheduling.

Geographic Load Shifting:

- Extending the framework to multi-region data centers would allow workloads to be shifted across locations with lower carbon intensity, further improving sustainability.

Advanced Deep Learning Models:

- Incorporating models such as LSTM, GRU, or Transformers could capture complex temporal dependencies and improve forecasting accuracy.

Multi-Objective Optimization:

- Future systems can jointly optimize carbon emissions, job delay, energy cost, and system performance using advanced optimization techniques.

Adaptive Conformal Prediction:

- Developing dynamic calibration strategies for conformal prediction can improve reliability under changing workload patterns.

Real-Time System Deployment:

- Integrating the framework into live data center scheduling systems would allow evaluation under real operational constraints.

4.3 References

1. Hall, S., et al. (2021). Carbon-intelligent computing for datacenters. Google Research. This work introduces a production-scale system that shifts flexible workloads away from periods of high carbon intensity using Virtual Capacity Curves (VCC) and Distributionally Robust Optimization (DRO). The system applies Conditional Value-at-Risk (CVaR) constraints and Wasserstein ambiguity sets to provide probabilistic guarantees on performance while reducing emissions in large-scale data center infrastructure.
2. Zhang, Y., et al. (2024). Cloud computing load prediction method based on CNN-BiLSTM model under low-carbon background. Scientific Reports. The study proposes a hybrid CNN–BiLSTM architecture for predicting cloud workload demand using the Google Cluster Trace dataset. CNN is used for spatial feature extraction and BiLSTM for temporal sequence modeling. The predicted CPU utilization is converted into energy consumption

and carbon emissions using a dynamic server power model, achieving significant error reduction compared with traditional models.

3. Yu, X., et al. (2025). Dual-splitting conformal prediction for multi-step time series forecasting. Applied Soft Computing. This research proposes a Dual-Splitting Conformal Prediction (DSCP) framework that generates asymmetric prediction intervals for time-series forecasting. The method uses k-means clustering with soft-DTW and Kolmogorov–Smirnov tests to improve prediction interval accuracy, demonstrating improved Winkler Scores and showing potential carbon reduction benefits in simulated data center optimization scenarios.

4.4 Appendix

Notebook 1

Cell 1 – Install Required Libraries

```
import subprocess, sys
subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "pyspark", "pyarrow", "pandas"])
```

Cell 2 – Import Libraries and Setup Environment

```
import os
import json
import warnings
import shutil
import urllib.request
warnings.filterwarnings('ignore')

HADOOP_HOME = os.path.join(os.getcwd(), "hadoop")
HADOOP_BIN = os.path.join(HADOOP_HOME, "bin")
os.makedirs(HADOOP_BIN, exist_ok=True)
```

Cell 3 – Import PySpark Modules

```
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql import functions as F
```

Cell 4 – Start Spark Session

```
spark = SparkSession.builder \
.appName("Cluster ETL Pipeline") \
```

```
.getOrCreate()
```

Cell 5 – Define Dataset Schema

```
schema = StructType([
StructField("machine_id", StringType(), True),
StructField("time_stamp", LongType(), True),
StructField("cpu_util_percent", DoubleType(), True),
StructField("mem_util_percent", DoubleType(), True),
StructField("mem_gps", DoubleType(), True),
StructField("mkpi", DoubleType(), True),
StructField("net_in", DoubleType(), True),
StructField("net_out", DoubleType(), True),
StructField("disk_io_percent", DoubleType(), True)
])
```

Cell 6 – Load Dataset

```
data_path = "data/raw/cluster_data.csv"

raw_df = spark.read.csv(data_path, header=True, schema=schema)
```

Cell 7 – Show Sample Data

```
raw_df.show(5)
```

Cell 8 – Basic Dataset Statistics

```
raw_df.describe().show()
```

Cell 9 – Null Value Counts

```
null_counts = raw_df.select(
[F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c) for c in raw_df.columns]
)

null_counts.show()
```

Cell 10 – Get Timestamp Range

```
ts_stats = raw_df.agg(
F.min("time_stamp").alias("ts_min"),
F.max("time_stamp").alias("ts_max")
).collect()[0]
```

Cell 11 – Identify Invalid Values

```
METRIC_COLS =
["cpu_util_percent","mem_util_percent","mem_gps","mkpi","net_in","net_out","disk_io_per
cent"]

for col_name in METRIC_COLS:
neg_count = raw_df.filter(F.col(col_name) < 0).count()
over_count = raw_df.filter(F.col(col_name) > 100).count()
if neg_count > 0 or over_count > 0:
    print(col_name, neg_count, over_count)
```

Cell 12 – Replace Invalid Values With Null

```
clean_df = raw_df

for col_name in METRIC_COLS:
clean_df = clean_df.withColumn(
col_name,
F.when((F.col(col_name) < 0) | (F.col(col_name) > 100), None).otherwise(F.col(col_name))
)
```

Cell 13 – Create Time Buckets

```
bucketed_df = clean_df.withColumn(
"ts_bucket",
(F.floor(F.col("time_stamp") / 300) * 300).cast(LongType())
)
```

Cell 14 – Aggregate Metrics

```
agg_exprs = [F.mean(c).alias(c) for c in METRIC_COLS]

agg_df = bucketed_df.groupBy("machine_id","ts_bucket").agg(*agg_exprs)
```

Cell 15 – Sort Data

```
agg_df = agg_df.orderBy("machine_id","ts_bucket")
```

Cell 16 – Cache Aggregated Data

```
agg_df.cache()
```

Cell 17 – Compute Machine Completeness

```
completeness = agg_df.filter(F.col("cpu_util_percent").isNotNull()) \
.groupBy("machine_id") \
.agg(
F.count("*").alias("n_records"),
F.mean("cpu_util_percent").alias("avg_cpu"),
F.stddev("cpu_util_percent").alias("std_cpu")
) \
.orderBy(F.desc("n_records"))
```

Cell 18 – Show Top Machines

```
completeness.show(20)
```

Cell 19 – Select Top Machines

```
TOP_N = 10

top_machines = [row["machine_id"] for row in completeness.limit(TOP_N).collect()]
```

Cell 20 – Filter Selected Machines

```
selected_df = agg_df.filter(F.col("machine_id").isin(top_machines))
```

Cell 21 – Export Data

```
OUT_DIR = "data/processed"

full_out = os.path.join(OUT_DIR, "full")
subset_out = os.path.join(OUT_DIR, "subset")

for path in [full_out, subset_out]:
if os.path.exists(path):
shutil.rmtree(path)

agg_df.write.parquet(full_out)
selected_df.write.parquet(subset_out)
```

Cell 22 – Create Summary Table

```
import pandas as pd

summary = pd.DataFrame({
```

```
"Metric": ["Total Rows Raw","Rows After Cleaning","Aggregated Rows","Selected Machines"],
"Value": [raw_df.count(), clean_df.count(), agg_df.count(), TOP_N]
})
```

Cell 23 – Save Metadata

```
meta_path = os.path.join(OUT_DIR,"etl_metadata.json")

etl_summary = {
"selected_machines": top_machines,
"selected_machine_count": TOP_N
}

with open(meta_path,"w") as f:
json.dump(etl_summary,f,indent=2)
```

Cell 24 – Stop Spark Session

```
spark.stop()

print("ETL Complete")
```

Notebook 2

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pyarrow.parquet as pq
import warnings
warnings.filterwarnings("ignore")
```

Cell 2 – Load Processed Data

```
data_path = "data/processed/subset"

df = pd.read_parquet(data_path)

print(df.shape)
df.head()
```

Cell 3 – Convert Timestamp to Datetime

```
df["datetime"] = pd.to_datetime(df["ts_bucket"], unit="s")

df = df.sort_values(["machine_id", "datetime"])

df.head()
```

Cell 4 – Select CPU Utilization Data

```
cpu_df = df[["machine_id", "datetime", "cpu_util_percent"]]

cpu_df.head()
```

Cell 5 – Convert Long Format to Wide Format

```
cpu_wide = cpu_df.pivot(index="datetime", columns="machine_id", values="cpu_util_percent")

cpu_wide.head()
```

Cell 6 – Handle Missing Values

```
cpu_wide = cpu_wide.fillna(method="ffill")
cpu_wide = cpu_wide.fillna(method="bfill")

cpu_wide.head()
```

Cell 7 – Compute Cluster Average CPU

```
cpu_wide["cpu_cluster_avg"] = cpu_wide.mean(axis=1)

cpu_wide.head()
```

Cell 8 – Generate Synthetic Carbon Intensity Signal

```
time_index = np.arange(len(cpu_wide))

carbon_signal = 400 + 100*np.sin(2*np.pi*time_index/288) + np.random.normal(0,10,len(cpu_wide))

cpu_wide["carbon_intensity"] = carbon_signal
```

Cell 9 – Normalize Features

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

scaled_values = scaler.fit_transform(cpu_wide)

scaled_df = pd.DataFrame(scaled_values, columns=cpu_wide.columns, index=cpu_wide.index)

scaled_df.head()
```

Cell 10 – Plot Cluster CPU Utilization

```
plt.figure(figsize=(12,5))
plt.plot(cpu_wide.index, cpu_wide["cpu_cluster_avg"])
plt.title("Average Cluster CPU Utilization")
plt.xlabel("Time")
plt.ylabel("CPU Utilization")
plt.show()
```

Cell 11 – Plot Carbon Intensity

```
plt.figure(figsize=(12,5))
plt.plot(cpu_wide.index, cpu_wide["carbon_intensity"])
```

```
plt.title("Carbon Intensity Signal")
plt.xlabel("Time")
plt.ylabel("Carbon Intensity")
plt.show()
```

Cell 12 – Correlation Heatmap

```
plt.figure(figsize=(10,8))
sns.heatmap(cpu_wide.corr(), cmap="coolwarm")
plt.title("Feature Correlation Matrix")
plt.show()
```

Cell 13 – Save Final Time Series Dataset

```
output_path = "data/timeseries_dataset.csv"

scaled_df.to_csv(output_path)

print("Time series dataset saved.")
```

Cell 14 – Dataset Summary

```
print("Total time points:", len(cpu_wide))
print("Total machines:", cpu_wide.shape[1]-2)
print("Features:", cpu_wide.columns)
```

Notebook 3

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import STL

import warnings
```

```
warnings.filterwarnings("ignore")
```

Cell 2 – Load Time Series Dataset

```
data_path = "data/timeseries_dataset.csv"

df = pd.read_csv(data_path, index_col=0, parse_dates=True)

df.head()
```

Cell 3 – Select Target Variable

```
cpu_series = df["cpu_cluster_avg"]

cpu_series.head()
```

Cell 4 – Plot CPU Time Series

```
plt.figure(figsize=(12,5))
plt.plot(cpu_series)
plt.title("Cluster CPU Utilization Over Time")
plt.xlabel("Time")
plt.ylabel("CPU Utilization")
plt.show()
```

Cell 5 – Augmented Dickey-Fuller Test

```
adf_result = adfuller(cpu_series)

print("ADF Statistic:", adf_result[0])
print("p-value:", adf_result[1])
```

Cell 6 – Differencing Time Series

```
cpu_diff = cpu_series.diff().dropna()

plt.figure(figsize=(12,5))
plt.plot(cpu_diff)
plt.title("Differenced CPU Series")
plt.show()
```

Cell 7 – ADF Test After Differencing

```
adf_diff = adfuller(cpu_diff)

print("ADF Statistic:", adf_diff[0])
print("p-value:", adf_diff[1])
```

Cell 8 – STL Decomposition

```
stl = STL(cpu_series, period=288)

result = stl.fit()

result.plot()
plt.show()
```

Cell 9 – Plot Autocorrelation Function

```
plt.figure(figsize=(10,5))
plot_acf(cpu_series, lags=50)
plt.show()
```

Cell 10 – Plot Partial Autocorrelation Function

```
plt.figure(figsize=(10,5))
plot_pacf(cpu_series, lags=50)
plt.show()
```

Cell 11 – Distribution Analysis

```
plt.figure(figsize=(8,5))
sns.histplot(cpu_series, kde=True)

plt.title("Distribution of CPU Utilization")
plt.show()
```

Cell 12 – Box Plot

```
plt.figure(figsize=(6,4))
sns.boxplot(x=cpu_series)

plt.title("CPU Utilization Boxplot")
plt.show()
```

Cell 13 – Rolling Mean and Standard Deviation

```
rolling_mean = cpu_series.rolling(window=50).mean()
rolling_std = cpu_series.rolling(window=50).std()

plt.figure(figsize=(12,5))
plt.plot(cpu_series, label="Original")
plt.plot(rolling_mean, label="Rolling Mean")
plt.plot(rolling_std, label="Rolling Std")

plt.legend()
plt.title("Rolling Statistics")
plt.show()
```

Notebook 4

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import train_test_split

from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.regime_switching.markov_autoregression import MarkovAutoregression

import warnings
warnings.filterwarnings("ignore")
```

Cell 2 – Load Dataset

```
data_path = "data/timeseries_dataset.csv"

df = pd.read_csv(data_path, index_col=0, parse_dates=True)

df.head()
```

Cell 3 – Select Target and Feature Variables

```
y = df["cpu_cluster_avg"]
X = df[["carbon_intensity"]]
```

Cell 4 – Train Test Split

```
train_size = int(len(df) * 0.7)
val_size = int(len(df) * 0.15)

train = df[:train_size]
val = df[train_size:train_size+val_size]
test = df[train_size+val_size:]
```

Cell 5 – Prepare Training Variables

```
y_train = train["cpu_cluster_avg"]
y_test = test["cpu_cluster_avg"]
```

```
X_train = train[["carbon_intensity"]]
X_test = test[["carbon_intensity"]]
```

Cell 6 – SARIMAX Model Training

```
sarimax_model = SARIMAX(y_train, exog=X_train, order=(2,1,2), seasonal_order=(1,1,1,288))

sarimax_result = sarimax_model.fit()
```

Cell 7 – SARIMAX Forecast

```
sarimax_forecast = sarimax_result.predict(start=len(y_train), end=len(y_train)+len(y_test)-1, exog=X_test)
```

Cell 8 – Calculate SARIMAX Metrics

```
mae_sarimax = mean_absolute_error(y_test, sarimax_forecast)
rmse_sarimax = np.sqrt(mean_squared_error(y_test, sarimax_forecast))

print("SARIMAX MAE:", mae_sarimax)
print("SARIMAX RMSE:", rmse_sarimax)
```

Cell 9 – Plot SARIMAX Forecast

```
plt.figure(figsize=(12,5))
plt.plot(y_test.values, label="Actual")
plt.plot(sarimax_forecast.values, label="Forecast")
plt.legend()
plt.title("SARIMAX Forecast vs Actual")
plt.show()
```

Cell 10 – Markov Switching Model

```
markov_model = MarkovAutoregression(y_train, k_regimes=2, order=1)
```

```
markov_result = markov_model.fit()
```

Cell 11 – Markov Model Prediction

```
markov_forecast = markov_result.predict(start=len(y_train), end=len(y_train)+len(y_test)-1)
```

Cell 12 – Markov Model Metrics

```
mae_markov = mean_absolute_error(y_test, markov_forecast)
rmse_markov = np.sqrt(mean_squared_error(y_test, markov_forecast))

print("Markov MAE:", mae_markov)
print("Markov RMSE:", rmse_markov)
```

Cell 13 – Plot Markov Forecast

```
plt.figure(figsize=(12,5))
plt.plot(y_test.values, label="Actual")
plt.plot(markov_forecast.values, label="Forecast")
plt.legend()
plt.title("Markov Switching Forecast")
plt.show()
```

Cell 14 – Model Comparison Table

```
results = pd.DataFrame({
"Model": ["SARIMAX","Markov"],
"MAE": [mae_sarimax, mae_markov],
"RMSE": [rmse_sarimax, rmse_markov]
})

results
```

Cell 15 – Save Forecast Results

```
forecast_df = pd.DataFrame({
"actual": y_test,
"sarimax": sarimax_forecast,
"markov": markov_forecast
})

forecast_df.to_csv("data/forecast_results.csv")

print("Forecast results saved.")
```

Notebook 5

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

Cell 2 – Load Forecast Data

```
data_path = "data/forecast_results.csv"

df = pd.read_csv(data_path, index_col=0, parse_dates=True)

df.head()
```

Cell 3 – Load Carbon Intensity Data

```
carbon_data = pd.read_csv("data/timeseries_dataset.csv", index_col=0, parse_dates=True)

df["carbon_intensity"] = carbon_data["carbon_intensity"]
```

Cell 4 – Define Power Model Parameters

```
P_idle = 120
P_max = 300
```

Cell 5 – Define Power Consumption Function

```
def power_consumption(cpu):
return P_idle + (P_max - P_idle) * cpu
```

Cell 6 – Compute Power Usage

```
df["power"] = df["sarimax"].apply(power_consumption)
```

Cell 7 – Compute Energy Consumption

```
df["energy"] = df["power"] * (5/60)
```

Cell 8 – Compute Carbon Emissions

```
df["co2_emissions"] = df["energy"] * df["carbon_intensity"] / 1000
```

Cell 9 – Define Job Parameters

```
total_jobs = 100
batch_ratio = 0.2

batch_jobs = int(total_jobs * batch_ratio)
online_jobs = total_jobs - batch_jobs
```

Cell 10 – Baseline FIFO Scheduling

```
fifo_emissions = df["co2_emissions"].sum()

print("FIFO emissions:", fifo_emissions)
```

Cell 11 – Carbon Aware Scheduling

```
low_carbon_threshold = df["carbon_intensity"].quantile(0.3)

green_slots = df[df["carbon_intensity"] <= low_carbon_threshold]

carbon_aware_emissions = green_slots["co2_emissions"].sum()

print("Carbon aware emissions:", carbon_aware_emissions)
```

Cell 12 – Risk Aware Scheduling

```
safe_threshold = 0.8

safe_slots = df[df["sarimax"] < safe_threshold]

risk_aware_emissions = safe_slots["co2_emissions"].sum()

print("Risk aware emissions:", risk_aware_emissions)
```

Cell 13 – Compare Scheduling Strategies

```
results = pd.DataFrame({
"Strategy": ["FIFO","Carbon Aware","Risk Aware"],
"CO2 Emissions": [fifo_emissions, carbon_aware_emissions, risk_aware_emissions]
})

results
```

Cell 14 – Plot Emission Comparison

```
plt.figure(figsize=(8,5))
```

```
plt.bar(results["Strategy"], results["CO2 Emissions"])
plt.title("CO2 Emissions Comparison")
plt.ylabel("Emissions")
plt.show()
```

Cell 15 – Emission Reduction Calculation

```
baseline = fifo_emissions
reduction = (baseline - risk_aware_emissions) / baseline * 100

print("Emission Reduction:", reduction, "%")
```

Cell 16 – Save Scheduling Results

```
results.to_csv("data/scheduling_results.csv")

print("Scheduling results saved.")
```

Notebook 6

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

import warnings
warnings.filterwarnings("ignore")
```

Cell 2 – Load Forecast Results

```
data_path = "data/forecast_results.csv"

df = pd.read_csv(data_path, index_col=0, parse_dates=True)
```

```
df.head()
```

Cell 3 – Select Actual and Predicted Values

```
y_true = df["actual"]
y_pred = df["sarimax"]
```

Cell 4 – Split Calibration and Test Data

```
cal_size = int(len(df) * 0.5)

y_cal = y_true[:cal_size]
y_cal_pred = y_pred[:cal_size]

y_test = y_true[cal_size:]
y_test_pred = y_pred[cal_size:]
```

Cell 5 – Compute Residual Errors

```
residuals = np.abs(y_cal - y_cal_pred)
```

Cell 6 – Compute Quantile for Prediction Interval

```
alpha = 0.05

q = np.quantile(residuals, 1 - alpha)

print("Quantile value:", q)
```

Cell 7 – Generate Prediction Intervals

```
lower_bound = y_test_pred - q
upper_bound = y_test_pred + q
```

Cell 8 – Store Interval Results

```
interval_df = pd.DataFrame({
"actual": y_test,
"prediction": y_test_pred,
"lower_bound": lower_bound,
"upper_bound": upper_bound
})

interval_df.head()
```

Cell 9 – Check Coverage

```
coverage = np.mean((y_test >= lower_bound) & (y_test <= upper_bound))

print("Prediction Interval Coverage:", coverage)
```

Cell 10 – Plot Prediction Intervals

```
plt.figure(figsize=(12,5))
plt.plot(y_test.values, label="Actual")
plt.plot(y_test_pred.values, label="Prediction")

plt.fill_between(range(len(lower_bound)), lower_bound, upper_bound, alpha=0.3)

plt.legend()
plt.title("Conformal Prediction Intervals")
plt.show()
```

Cell 11 – Interval Width Analysis

```
interval_width = upper_bound - lower_bound

print("Average Interval Width:", interval_width.mean())
```